

Lab 2 - Writing Publisher/Subscriber Nodes in ROS*

EECS/ME/BIOE C106A/206A Fall 2026

Goals

By the end of this lab you should be able to:

- Write ROS nodes in Python that both publish and subscribe to topics
 - Define custom ROS message types to exchange data between nodes
 - Create and build a new package with dependencies, source code, and message definitions
 - Understand the use and basic implementation of a service
 - Write a new node that interfaces with some existing ROS code
-

Note: Much of this lab is borrowed from [the official ROS tutorials](#). We picked out the material you will find most useful in this class, but feel free to explore other resources if you are interested in learning more.

Contents

1	Creating an Instructional Account	1
2	Connecting to the Lab Machines Remotely	2
3	Initial configuration	3
3.1	Running ROS 2 in a Container (Skip section 3.1 if you are doing this lab remote)	3
3.2	Workspace setup	4
4	Introduction	5
5	Write a controller for turtlesim	6
5.1	Guiding questions to help you get started:	6
6	Control Turtlesim Using Services	8

1 Creating an Instructional Account

To create a lab account, visit <https://acropolis.cs.berkeley.edu/account/webacct/> and click "Login using your Berkeley CalNet ID." On the next page, next to ee106a click "Create a new Account" and **save** this login information to use for your 106A labs! Make sure both partners do this so everyone has an account.

Be sure to use your ee106a account for all labs! This is because there are some things installed in our class-specific accounts that are not available in cs199-* accounts.

*Modified in August 2026 by Kourosh Salahi and Purva Marfatia. Rewritten and adapted for ROS2 in Summer 2025 by Jaeyun Stella Seo. Rewritten in Fall 2023 by Shrey Aeron. Developed by Aaron Bestick and Austin Buchan, Fall 2014.

2 Connecting to the Lab Machines Remotely

If you are working on one of the lab machines in person, then you can skip this part.

Start by going to <https://instrdp.eecs.berkeley.edu/> and logging in with your CalNet credentials.

Go to **Login Servers** > **derby**, and log in using the credentials you use on the lab computers (**ee106a-xxx**).

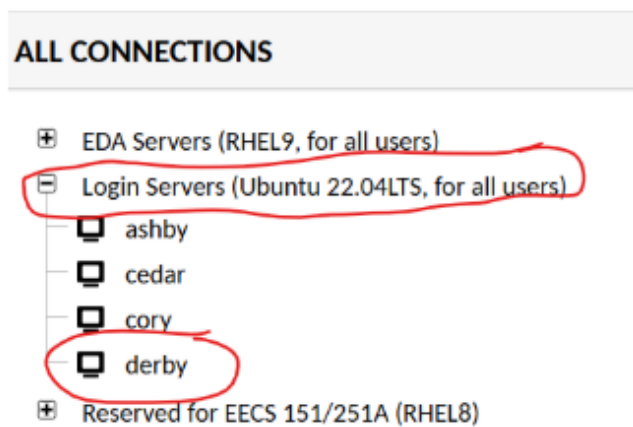


Figure 1: Selecting derby under Login Servers.

Open **MATE terminal** from the menu on the top left.

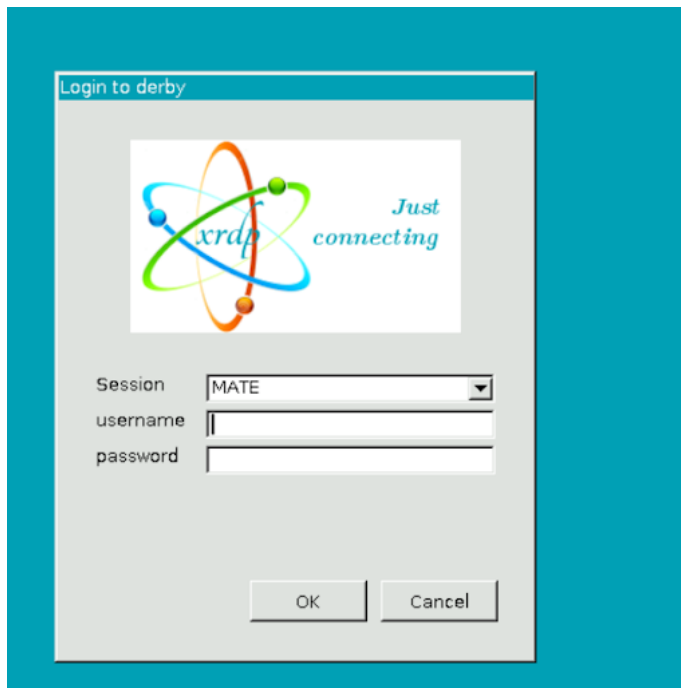


Figure 2: Opening MATE terminal from the top-left menu.

The first time you do this, initialize the container system for your account by running

```
inst-containers-setup
```

Next, create your lab container with

```
~ee106a/create-lab12-container
```

Now, enter your lab container using

```
distrobox enter lab12
```

```
derby [195] ~ # distrobox enter lab12
WARN[0000] "/" is not a shared mount, this could cause issues or missing mounts
with rootless containers
Starting container... [ OK ]
Installing basic packages... [ OK ]
Setting up devpts mounts... [ OK ]
Setting up read-only mounts... [ OK ]
Setting up read-write mounts... [ OK ]
Setting up host's sockets integration... [ OK ]
Integrating host's themes, icons, fonts... [ OK ]
Setting up distrobox profile... [ OK ]
Setting up sudo... [ OK ]
Setting up user's group list... [ OK ]

Container Setup Complete!
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

'ee106a-taj' is using 546/4194 MB (13%) of its disk quota on /home/shadow.
(Type 'more /share/b/pub/disk.quotas' for more information.)

Setup for ROS2 simulation for labs 1/2 only
lab12 [195] ~ # █
```

Figure 3: The lab12 container after entering the container.

From here, all instructions are the same. Just remember to run

```
distrobox enter lab12
```

Please note that you should only use the above command when working on the lab machines remotely. When you are physically on one of the lab desktops, you should still run **distrobox enter ros2**.

3 Initial configuration

Since we started Lab 1 in a VM, we have moved the lab machine setup instructions here. Please follow these instructions as all labs and the final project for the rest of the semester will depend on the lab machines. For labs 1 and 2 ONLY, if you would like to complete them remotely, please see section 2 of this document.

3.1 Running ROS 2 in a Container (Skip section 3.1 if you are doing this lab remote)

Instead of installing ROS 2 directly on your computer, we'll run it inside a *container*. Think of a container as a sealed lunchbox for software: everything you need goes inside, and it won't spill out or make a mess on your computer. We have packaged Ubuntu, ROS 2, and other tools you'll need into this "box". When you open this box, it will work exactly the same way every time. And when you're done and close the box, the rest of your computer is untouched.

So why use a container? There are a few reasons:

- **Same Setup for Everyone:** No matter which lab machine you use, you'll have the exact same ROS 2 environment.
- **No Risk to Your Computer:** You can't accidentally break anything on the lab PC.
- **Easy to Start Over:** If something goes wrong, just throw away the container and start a fresh one—no complicated fixes needed.
- **No Admin Rights Needed:** You don't need special permissions to install or update anything.

Now let's get this container on your account! Once you are logged into your class account, open up the Terminal and initialize the container system for your account by running

```
inst-containers-setup
```

You'll see "Container setup complete!"

Next, we'll create a ROS 2 container in Distrobox with the following command.

```
~ee106a/create-ros2-container
```

This should happen pretty quickly. Now, we'll enter our ROS 2 container using

```
distrobox enter ros2
```

The first time you do this, it will install a bunch of different things before telling you setup is complete. Look at the first part of your command line interface. You'll note that the hostname has changed from the computer you're using ("c105-n") to "ros2." This shows that you've now switched to being inside the container.

Note that the container doesn't mean your lab account has a "split brain" where what happens inside and outside the container are independent of each other. That's more akin to having a dual-boot computer, or a virtual machine within your computer. Instead of that, our container can interact with everything you would hope it could. So you can continue to write and edit files normally, and they will still be saved to your lab account.

Moving forward, if you want to use ROS in a Terminal, you will need to run first enter the container so that the Terminal has opened the lunchbox with ROS 2 in it.

```
distrobox enter ros2
```

3.2 Workspace setup

Throughout the semester, course staff will publish the starter code to fix bugs or release new labs. Clone the starter code repository by running the following command from your home folder (~):

```
cd ~  
git clone https://github.com/ucb-ee106/106a-student-labs-starter.git ros_workspaces
```

This clones the directory as "**ros_workspaces**" (we'll explain what this name means later), which currently should only contain a /lab0 workspace and a /lab1 workspace. We will be working in both workspaces for this lab. You will note that lab0 is empty other than a PDF of this document. This is intentional so that in the first part of this lab, you can see how to build a workspace package from scratch. In lab1 there is an existing package that you will play around with. Note that **ros_workspaces** will eventually contain several lab-specific workspaces (/lab2, /lab3, etc.).

Run the following command to rename the remote from "**origin**" to "**starter**".

```
cd ~/ros_workspaces  
git remote rename origin starter
```

Now, if you ever want to pull updated starter code, you'd execute the following command:

```
git pull starter main
```

Instructions to publish your lab code to your personal github repository are discussed at the end of the lab 1 document

Two other useful commands to know are **rmdir** to remove an empty directory and **rm -r** to remove a non-empty directory.

4 Introduction

In Lab 1, you were introduced to the concept of the ROS computation graph. The graph is populated with:

- **nodes:** an executable representing an individual ROS software process, typically performing some internal data processing and communicating with other nodes
- **topics:** nodes can *publish* to a topic to send messages or *subscribe* to a topic to receive messages
- **services:** nodes can request services offered by other nodes to receive a response

In Lab 2, you will explore how to write your own ROS nodes and enable them to communicate with each other by sending data through topics and services. Luckily, this process is fairly straightforward, as ROS provides a simple framework to construct these pipelines.

Before every lab, you should pull the updated starter code. In terminal, navigate to the `~/ros_workspaces` directory and ensure that you have pushed all Lab 1 changes to Github (as outlined in the Github section of Lab 1). Then, run

```
git pull starter main
```

to pull the lab 2 starter code.

5 Write a controller for turtlesim

Let's write a new controller for the Turtlesim node you used in the lab last week. This node will replace `turtle_teleop_key`. Since the `turtlesim` node is the subscriber in this example, you'll only need to write a single publisher node `turtle_controller`.

Create a new package `lab2_turtlesim` (that depends on `turtlesim` and other appropriate packages) to hold your node:

```
ros2 pkg create lab2_turtlesim --build-type ament_python --dependencies rclpy std_msgs geometry_msgs turtlesim
```

Your new node `turtle_controller` should do the following:

1. Accept a command line argument specifying the name of the turtle it should control. Running

```
ros2 run lab2_turtlesim turtle_controller <name_of_turtle>
```

will start a controller node that controls `<name_of_turtle>`. The Python package `sys` will help you get command line arguments (specifically, [sys.argv](#)).

2. Publish twist control messages on the appropriate topic (`ros2 topic list` could be useful) whenever the user presses certain keys on the keyboard, as in the original `turtle_teleop_key`. (It turns out that capturing individual keystrokes from the terminal is [slightly complicated](#) — it's a great bonus if you can figure it out, but feel free to use `input()` instead.)
3. Start `turtlesim` and spawn multiple turtles in it. See if you can open multiple instances of your new turtle controller node, each linked to a different turtle.

```
ros2 service call /spawn turtlesim/srv/Spawn \
    "{x: 2.0, y: 2.0, theta: 1.2, name: '<name_of_turtle>'}"
```

Note the name of the turtle when running `turtle_controller.py` should match `<name_of_turtle>`.

5.1 Guiding questions to help you get started:

- Start by running `ros2 run turtlesim turtlesim_node`, which you should recognize from Lab 1 as spawning a turtle!
- Recall: your controller needs to communicate to this node for it to move. What command can we use to find out what topic `turtlesim` is subscribed to?
- You need to write a **publisher**. Where have you seen this in Lab 1? Feel free to copy code over.
- Once you've got your publisher, what do you publish to it? Check the message type, and then look at this [reference](#).
- Possibly helpful information: since your turtle lives in a 2D world, it can only translate along x and y and only rotate about z.
- After importing it from `geometry_msgs`, you can create a `Twist` using `msg = Twist()` and then edit the attributes according to what directional command you want to give the turtle.
- It doesn't matter too much how you implement the mapping from keys to turtle directions, so long as you can demonstrate your turtle moving differently according to the keystrokes.
- It's fine to use Python's `input()` to record keystrokes.
- Use `self.get_logger().info('your log message')` if you want to log something for debugging!

Checkpoint 1

Submit a checkoff request at tinyurl.com/ee106a-fa26lab for a TA to come and checkoff your work. You should be able to:

- Show that you completed your Pre-Lab Quiz on Gradescope.
 - Explain all the contents of your `lab2_turtlesim` package
 - Show that your new package builds successfully
 - Demonstrate the functionality of your new turtle controller node by controlling two turtles with different inputs
-

6 Control Turtlesim Using Services

Now that we have learned how to use publishers and subscribers to control turtles in Turtlesim, we will explore an alternative communication pattern: services/servers/clients.

What is a ROS 2 Service?

A **service** is a one-time, **request–response** exchange between two nodes. A **client** node sends a **request**; a **server** node does some work and sends back a **response**. Use a service when you need information or want to trigger a quick action *on demand*, not continuously. Services in ROS 2 follow a call-and-response model; they are intended for relatively quick, one-off interactions. See the ROS 2 docs for details.

Typical flow:

1. **Client sends a request.** Your code fills out the request fields and sends it to the service.
2. **Server callback runs.** The service server node receives the request, runs its callback (does the work), and fills out a response.
3. **Client gets the response.** The client code receives the response (either by waiting for it or by checking later, depending on how you call the service) and continues.

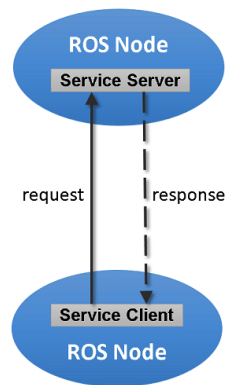


Figure 4: Service-based communication. Source: MathWorks

To define a service in ROS in Python, we must define three things: a service type, a server node, and a client node. For example, take a look at the package `turtle_patrol_interface` in `lab2`. Examine the `Patrol.srv` file, which should look like this:

```
float32 vel
float32 omega
---
geometry_msgs/Twist cmd
```

The file defines a new service type with a request and a response separated by a `---` line. The request consists of two `float32` messages: one for the translational velocity of the turtle and one for the angular velocity. The response consists of a `geometry_msgs/Twist` message in this example. The server uses this message to construct the velocity command that is published to the turtle's `/cmd_vel` topic.

The `patrol_server.py` file defines a node that provides the service. Look at this script to understand how the node works. The `patrol_client.py` file defines a node that uses the service. This file serves as an example of how services are called programmatically from a node.

Next, let us run the service as follows:

- Close all previous ROS nodes

- Open a new turtlesim node

```
ros2 run turtlesim turtlesim_node
```

- Run the server node

```
ros2 run turtle_patrol patrol_server
```

- Run the client node that calls the service

```
ros2 run turtle_patrol patrol_client
```

After successfully invoking the service, you will find that turtle1 starts to patrol in a circle!

You can also call the service from the command line with the command

```
ros2 service call /turtle1/patrol turtle_patrol_interface/srv/Patrol "{vel: 2.0, omega: 1.0}"
```

where you can replace the values for [vel] and [omega]. Try calling it a few times with different parameters to see how its behavior changes.

Now that you can control **one** turtle using a service, you will extend your system to control **multiple turtles** using **a single service server**. In addition, the service request must include a target pose, and the server must **teleport the requested turtle** before applying the patrol motion.

Example service call

```
ros2 service call /turtle_patrol turtle_patrol_interface/srv/Patrol \
"{turtle_name: '<name_of_turtle>', vel: 2.0, omega: 1.0, x: 1.0, y: 3.0, theta: 1.0}"
```

Requirements

Your new server node: multi_patrol_server

Create a new server node file named multi_patrol_server.py. Your multi_patrol_server must:

1. Control multiple turtles with one server process

- You must **not** run one server node per turtle.
- The service request contains **turtle_name**, and the server uses that field to decide which turtle to act on.

2. Teleportation support

- The service request includes **x**, **y**, **theta**.
- When a request arrives, the server must teleport that turtle to (**x**, **y**, **theta**) using the **turtlesim teleport service**, then begin commanding its patrol motion with the requested **vel** and **omega**.

What you need to change

1) Update the service definition

Create or modify a service type in **turtle_patrol_interface** so the request includes teleport inputs.

Suggested request fields

- string `turtle_name`
- float32 `vel`
- float32 `omega`
- float32 `x`
- float32 `y`
- float32 `theta`

You may choose the response fields (for example, `bool success` and `string message`) however you see fit.

Implementation hints

Ingest command line arguments (client)

Look back at the previous checkpoint to see how to parse multiple CLI arguments in ROS 2 Python nodes. Your client will need to parse:

```
ros2 run turtle_patrol multi_patrol_client <turtle_name> <x> <y> <theta> <velocity> <omega>
```

Discover the teleport service

You will teleport the turtle by calling an existing turtlesim service. To find the exact service name and type, use:

- `ros2 service list`
- `ros2 service type <service_name>`
- `ros2 interface show <service_type>`

These commands will lead you to the turtle's teleport service and its request fields. Your service should act as a client for the teleportation service, you can model it after `patrol_client`.

Keep per-turtle state in a dictionary

Because one server handles many turtles, you should maintain a dictionary keyed by `turtle_name`. This avoids spawning one server per turtle and keeps all turtle control in one place.

Examples of per-turtle state you may store:

- a publisher for `/<turtle_name>/cmd_vel`
- the `vel` & `omega` for this turtle

Iterate through your dictionary when actually publishing your command on a timer

Deliverable: Python client node

Create a client node file named `multi_patrol_client.py` that calls `/turtle_patrol` like this:

```
ros2 run turtle_patrol multi_patrol_client <turtle_name> <x> <y> <theta> <velocity> <omega>
```

The client must:

- parse these command line inputs
- populate the `Patrol` request
- call the service and print the response

Have fun building an army of patrolling turtles!

Next lab, we will start using the *real* robots!

Checkpoint 2

Submit a checkoff request at tinyurl.com/ee106a-fa26lab for a TA to come and check off your work. You should be able to:

- Explain how services differ from topics in a ROS node, and when each should be used.
 - Show that your modified service can make multiple turtles patrol in circles from different initial poses. Explain why your architecture works.
 - Push finished code to Github
-

Congrats on finishing Lab 2!